



Fence API — Client Integration Guide

Audience: Technical teams integrating with Fence for asset and payment transaction declaration.

API Version: v2

1. Getting Started

[1.1 Environments](#)

[1.2 Facility Scope](#)

[1.3 Credentials](#)

2. Authentication

[Obtaining an Access Token](#)

[Using the Token](#)

[Authorization Model](#)

3. Asset Declaration

[3.1 Overview](#)

[3.2 Step-by-Step Declaration Flow](#)

[Step 1: Create a Portfolio Declaration](#)

[Step 2: Add Assets to the Declaration](#)

[Step 3: Trigger the Declaration](#)

[3.3 Fast-Track Declaration Flow](#)

[3.4 Monitoring Declaration Progress](#)

[List Declaration Runs for a Deal](#)

[Get a Specific Declaration Run](#)

[List Per-Asset Results](#)

[Get a Single Asset Result](#)

[3.5 Dry Run Mode](#)

4. Document Declaration

[4.1 Document Registration](#)

[4.2 Upload Status](#)

- 5. [Payment Transaction Declaration](#)
 - 5.1 [Bulk Create](#)
 - [Transaction Types](#)
 - [Transaction Fields](#)
 - [Metadata by Transaction Type](#)
- 6. [Schemas Reference](#)
 - [Asset Declaration — Request](#)
 - [Asset Declaration — Fast Track Request](#)
 - [Portfolio Declaration Response](#)
 - [Declaration Run Response](#)
 - [Per-Asset Result Response](#)
 - [Supported Currencies](#)
- 7. [Error Handling](#)
 - [HTTP Status Codes](#)
 - [Pagination](#)
 - [Idempotency](#)
 - [Recommended Polling Strategy](#)

1. Getting Started

1.1 Environments

Fence provides two environments:

Environment	Base URL	Purpose
Sandbox	<code>https://dev-api.fence.finance</code>	Integration testing. Non-production data; safe to experiment.
Production	<code>https://api.fence.finance</code>	Live environment.

All API paths in this guide are relative to the environment base URL. For example, `/v2/{deal_id}/documents` becomes `https://dev-api.fence.finance/v2/{deal_id}/documents` in Sandbox.

1.2 Facility Scope

Each facility is identified in Fence by a `deal_id` — a UUID that scopes requests to that facility. Most endpoints include `{deal_id}` as a path parameter. Fence will

provide separate `deal_id` values for the **Sandbox** and **Production** environments during onboarding.

The login endpoint (`POST /v1/login/access-token`) is the exception: it does **not** take a `deal_id` . Authentication is performed with `client_id` + `client_secret` , and the returned token is implicitly scoped to the facilities your account is linked to.

1.3 Credentials

API credentials (`client_id` + `client_secret`) and your `deal_id` are provisioned by Fence per environment. To request them, contact your Fence onboarding representative. Fence will provide the credentials in a secure way during onboarding.

2. Authentication

Obtaining an Access Token

Fence uses **OAuth 2.0**.



The login endpoint does **not** require a `deal_id` . Authentication is performed with `client_id` + `client_secret` , and the returned token is implicitly scoped to the deals your account is linked to.

Client Credentials Grant (recommended for machine-to-machine integrations):

```
POST /v1/login/access-token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id={your_client_id}
&client_secret={your_client_secret}
```

Response:

```
{
  "access_token": "eyJraWQiOi...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Using the Token

Include the token in every subsequent request:

```
Authorization: Bearer {access_token}
```

Authorization Model

- Clients are authorized at the **deal level**. Your credentials grant access only to the deals your account is linked to.
- **Actions:** `read`, `write`, `create`, `declare` — each endpoint requires a specific action.
- Attempting to access a deal you're not authorized for returns `403 Forbidden`.

3. Asset Declaration

3.1 Overview

Asset declaration is the process of submitting your portfolio of assets (e.g., loans, receivables, advances) to Fence. Fence validates, persists, and processes these assets through a configurable pipeline that can include eligibility checks, borrow base calculations, and expected payment generation.

Two approaches are available:

Approach	Best For	Endpoints
Step-by-step	Large portfolios, incremental asset addition	<code>/v2/portfolio_declarations/</code>

Approach	Best For	Endpoints
Fast-track	Simpler integrations, single-call declaration	<code>/v2/declarations/</code>

Both approaches trigger the same processing pipeline and produce the same results.

3.2 Step-by-Step Declaration Flow

This approach gives you fine-grained control over the declaration lifecycle.

Step 1: Create a Portfolio Declaration

```
POST /v2/portfolio_declarations/
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "name": "Daily Portfolio Snapshot 2026-04-07",
  "deal_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Response (201 Created):

```
{
  "id": "d4e5f6a7-b8c9-0123-4567-890abcdef012",
  "deal_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "name": "Daily Portfolio Snapshot 2026-04-07",
  "asset_count": null,
  "created_at": "2026-04-07T10:00:00Z",
  "updated_at": "2026-04-07T10:00:00Z"
}
```

Step 2: Add Assets to the Declaration

You can call this endpoint multiple times to add assets in batches.

```
POST /v2/portfolio_declarations/{portfolio_declaration_id}/as
sets
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body (list of assets):

```
[
  {
    "external_id": "ADV-2026-001234",
    "contract_external_id": null,
    "properties": {
      "origination_installment_id": "INS-78901",
      "organization_name": "Acme Corp",
      "employer_name": "Acme Corp",
      "employee_id": "EMP-456",
      "user_state": "CA",
      "user_country": "US",
      "earned_wage": 2500.00,
      "total_principal_amount": 800.00,
      "total_fee_amount": 12.00,
      "outstanding_principal_amount": 800.00,
      "outstanding_fee_amount": 12.00,
      "outstanding_amount": 812.00,
      "transaction_date": "2026-04-01",
      "due_date": "2026-04-15",
      "currency": "USD",
      "status": "active"
    }
  }
]
```

Key points:

- `external_id` — Your unique identifier for the asset. Used for idempotency (upsert on re-declaration).
- `contract_external_id` — Optional. Links the asset to a specific contract. If `null`, Fence may auto-create a default umbrella contract (deal-dependent).
- `properties` — Deal-specific fields. The exact schema is agreed upon during onboarding and validated by Fence during the declaration pipeline.

Step 3: Trigger the Declaration

```
POST /v2/portfolio_declarations/{portfolio_declaration_id}/declare
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "dry_run": false
}
```

Response (202 Accepted):

```
{
  "id": "b1c2d3e4-f5a6-7890-bcde-f01234567890",
  "status": "QUEUED",
  "dry_run": false
}
```

The `202 Accepted` status indicates the declaration has been queued for asynchronous processing. Use the `id` to monitor progress.

3.3 Fast-Track Declaration Flow

A single endpoint that creates a portfolio declaration, adds assets, and triggers the pipeline in one call.

```
POST /v2/declarations/  
Authorization: Bearer {token}  
Content-Type: application/json
```

Request Body:

```
{  
  "deal_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",  
  "dry_run": false,  
  "assets": [  
    {  
      "external_id": "ADV-2026-001234",  
      "properties": {  
        "origination_installment_id": "INS-78901",  
        "organization_name": "Acme Corp",  
        "employee_id": "EMP-456",  
        "user_state": "CA",  
        "user_country": "US",  
        "total_principal_amount": 800.00,  
        "total_fee_amount": 12.00,  
        "outstanding_amount": 812.00,  
        "transaction_date": "2026-04-01",  
        "due_date": "2026-04-15",  
        "currency": "USD",  
        "status": "active"  
      }  
    }  
  ]  
}
```

Batching recommendation: The declaration queue processes declarations for a given deal sequentially. Each declaration has fixed infrastructure overhead, so sending many single-asset declarations will create significant backlog. Instead, batch assets over a period (e.g., every 15 minutes, hourly, or daily depending on your reporting needs) and send them in a single declaration.

3.4 Monitoring Declaration Progress

List Declaration Runs for a Deal

```
GET /v2/declarations/?deal_id={deal_id}&page=1&size=20
Authorization: Bearer {token}
```

Declaration Run Statuses:

Status	Description
QUEUED	Declaration received, waiting for processing.
PROCESSING	Pipeline is actively processing assets.
COMPLETED	All pipeline steps finished successfully. All assets are declarable.
COMPLETED_WITH_ERRORS	Pipeline finished but some assets are not declarable (e.g., failed eligibility). Check per-asset results.
FAILED	Pipeline encountered a critical error. Check error_details .

Get a Specific Declaration Run

```
GET /v2/declarations/{run_id}
Authorization: Bearer {token}
```

List Per-Asset Results

```
GET /v2/declarations/{run_id}/assets?page=1&size=50
Authorization: Bearer {token}
```

Per-Asset Result Statuses:

Status	Meaning
SUCCESSFUL	Asset passed this pipeline step.

Status	Meaning
UNSUCCESSFUL	Asset was processed but did not meet the criteria (e.g., failed eligibility). Still created but flagged.
FAILED	An error occurred processing this asset at this step.

Pipeline Steps:

Step Name	Description
ASSET_VERIFICATION	Validates asset against deal-specific eligibility criteria.
BORROW_BASE_VERIFICATION	Evaluates asset for borrow base inclusion (if applicable).
ASSET_CREATION	Creates or updates the asset in Fence. <code>details.operation</code> is <code>created</code> or <code>updated</code> .

Get a Single Asset Result

```
GET /v2/declarations/{run_id}/assets/{asset_id}
Authorization: Bearer {token}
```

3.5 Dry Run Mode

Both declaration flows support `dry_run: true`. In dry-run mode:

- The full validation pipeline runs (schema validation, eligibility checks, borrow base).
- **No data is persisted** — assets are not created/updated, no expected payments are generated.
- Results are still available via the monitoring endpoints.
- Use this to validate your data before committing.

Recommended workflow:

1. Submit a declaration with `dry_run: true`.
2. Inspect per-asset results for validation issues.
3. Fix any issues in your source data.

4. Submit again with `dry_run: false`.

4. Document Declaration

For deals that require verification, Fence validates each receivable against supporting portfolio documents (e.g., loan agreements, repayment authorizations, invoices). Clients register documents in batch, upload the files to S3 using the returned presigned URLs, and Fence processes them asynchronously.



`document_external_id` is the join key Fence uses to reconcile each document with its declared asset at verification time. The value convention (e.g., matching a specific asset field) is agreed during onboarding.

4.1 Document Registration

Register one or more documents in a single call. Fence creates the document records and returns presigned S3 URLs the client PUTs the file bytes to.

```
POST /v2/{deal_id}/documents
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "documents": [
    {
      "document_category": "loan_agreement",
      "document_external_id": "DOC-2026-001234",
      "file_name_with_ext": "agreement.pdf",
      "title": "Loan Agreement - Acme Borrower"
    }
  ]
}
```

Request Field Reference:

Field	Type	Required	Description
<code>document_category</code>	string	Yes	Category of the document (e.g., <code>loan_agreement</code> , <code>invoice</code> , <code>repayment_authorization</code>). Agreed during onboarding.
<code>document_external_id</code>	string	Yes	Client-provided unique identifier. Used as the join key to reconcile this document with its declared asset during verification; the specific value convention is agreed during onboarding.
<code>file_name_with_ext</code>	string	Yes	Filename including extension (e.g., <code>agreement.pdf</code>).
<code>title</code>	string	No	Human-readable title. Defaults to filename if omitted.

Response (201 Created):

```
{
  "documents": [
    {
      "document_external_id": "DOC-2026-001234",
      "presigned_url": "https://s3.amazonaws.com/...",
      "expires_at": "2026-04-07T11:00:00Z",
      "uploaded_at": null
    }
  ]
}
```

For each document in the response, PUT the file bytes to the returned `presigned_url` before `expires_at`. Presigned URLs are valid for 1 hour from issuance.

Re-registering with an existing `document_external_id` returns `422 Unprocessable Entity`. To retry an upload whose URL has expired or whose PUT failed, use Section 4.2 Upload Status to request a fresh presigned URL — do not re-register the document.

4.2 Upload Status

Use this endpoint for two purposes: (1) confirming Fence has received each file from S3, and (2) retrieving a fresh presigned URL for any document that has not yet been uploaded.

`uploaded_at` is populated once Fence registers the S3 event. Until then, each call to this endpoint returns a newly-signed `presigned_url` with a fresh `expires_at` one hour in the future, so you can safely retry uploads that failed or whose URL has expired without re-registering the document.

```
GET /v2/{deal_id}/documents?document_external_ids=DOC-2026-001234,DOC-2026-001235
Authorization: Bearer {token}
```

Response (200 OK):

```
{
  "documents": [
    {
      "document_external_id": "DOC-2026-001234",
      "presigned_url": null,
      "expires_at": null,
      "uploaded_at": "2026-04-07T10:15:32Z"
    },
    {
      "document_external_id": "DOC-2026-001235",
      "presigned_url": "https://s3.amazonaws.com/...",
      "expires_at": "2026-04-07T11:00:00Z",
      "uploaded_at": null
    }
  ]
}
```

```
]
}
```

Two mutually exclusive states are returned per document:

State	Fields populated	Meaning
Pending	<code>presigned_url</code> • <code>expires_at</code> ; <code>uploaded_at</code> is <code>null</code>	Document registered but not yet uploaded. The URL is freshly-signed on every call — retry as needed.
Uploaded	<code>uploaded_at</code> ; <code>presigned_url</code> and <code>expires_at</code> are <code>null</code>	File has been received by Fence. No new URL will ever be issued for this document — the record is now immutable for verification.

Unrecognized `document_external_id` values are silently omitted from the response (no error). The endpoint is safe to call repeatedly; it has no side effects on the document record.

5. Payment Transaction Declaration

Payment transactions represent financial movements associated with a deal — intakes (incoming payments), expenses, repayments, outbound transfers, dispositions, and more.

Base URL: All payment transaction endpoints are scoped to a deal:

```
/v2/{deal_id}/payments
```

5.1 Bulk Create

Create multiple payment transactions in a single request. The processing mode is controlled by the `dry_run` query parameter:

```
POST /v2/{deal_id}/payments/bulk?dry_run=false
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "data": [
    {
      "external_id": "TXN-20260407-001",
      "amount": 812.00,
      "transaction_date": "2026-04-07T14:30:00Z",
      "currency": "USD",
      "transaction_type": "intake",
      "from_": "Borrower Account",
      "to_": "SPV Collection Account",
      "memo": "Repayment for advance ADV-2026-001234",
      "bank_account_id": "e5f6a7b8-c9d0-1234-ef56-7890abcdef0
1",
      "metadata": {
        "metadata_type": "intake",
        "type": "repayment_collection"
      }
    }
  ]
}
```

Query Parameters:

Parameter	Type	Default	Description
<code>dry_run</code>	bool	<code>true</code>	If <code>true</code> , validates without persisting (synchronous 200). If <code>false</code> , validates and enqueues for async processing (202). Set to <code>false</code> explicitly to create transactions.

Transaction Types

Type	Description	Metadata Type
<code>intake</code>	Incoming payment to the deal (collections, deposits)	<code>intake</code>

Type	Description	Metadata Type
<code>intake_meta</code>	Metadata-only intake (no financial movement)	<code>intake_meta</code>
<code>expense</code>	Fee or expense charged to the deal	<code>expense</code>
<code>repayment</code>	Repayment from borrower to lender	<code>repayment</code>
<code>outbound</code>	Outgoing payment from the deal	<code>outgoing_type</code>
<code>disposition</code>	Asset disposition event	—
<code>borrow</code>	Borrowing/draw event	<code>borrow</code>
<code>utilization</code>	Facility utilization event	<code>utilization</code>
<code>waterfall_payment</code>	Waterfall distribution payment	—
<code>outgoing_type</code>	General outgoing payment	<code>outgoing_type</code>

Transaction Fields

Field	Type	Required	Description
<code>external_id</code>	string	Yes	Your unique identifier for this transaction.
<code>amount</code>	float	Yes	Transaction amount.
<code>transaction_date</code>	datetime	Yes	When the transaction occurred.
<code>currency</code>	string	Yes	ISO currency code (e.g., <code>"USD"</code>).
<code>transaction_type</code>	string	No	One of the transaction types above.
<code>from_</code>	string	No	Source account identifier.
<code>to_</code>	string	No	Destination account identifier.
<code>memo</code>	string	No	Description or notes.
<code>bank_account_id</code>	UUID	No	Associated bank account in Fence.
<code>iban</code>	string	No	Recipient IBAN.
<code>routing_number</code>	string	No	Recipient routing number (when no IBAN).
<code>account_number</code>	string	No	Recipient account number (when no IBAN).

Field	Type	Required	Description
<code>metadata</code>	object	No	Type-specific metadata (see below).

Metadata by Transaction Type

Intake:

```
{ "metadata_type": "intake", "type": "repayment_collection" }
```

Expense:

```
{
  "metadata_type": "expense",
  "category": "servicing",
  "description": "Monthly servicing fee"
}
```

Repayment:

```
{ "metadata_type": "repayment", "repayment_type": "scheduled"
}
```

Borrow:

```
{
  "metadata_type": "borrow",
  "borrow_type": "initial_draw",
  "tranche_name": "Tranche A"
}
```

Utilization:

```
{
  "metadata_type": "utilization",
}
```

```

    "utilization_type": "draw",
    "tranche_name": "Tranche A"
  }

```

Outgoing Type:

```

{
  "metadata_type": "outgoing_type",
  "outgoing_type": "distribution",
  "payment_date": "2026-04-07T00:00:00Z"
}

```

All metadata types accept an optional `deal_specific_metadata` dict for custom fields.

6. Schemas Reference

Asset Declaration — Request

```

CreatePortfolioDeclarationAssetItem:
  external_id: string (required, min 1 char)
  contract_external_id: string | null
  properties: object | null (deal-specific fields)

```

Asset Declaration — Fast Track Request

```

FastTrackDeclarationApiInput:
  deal_id: UUID (required)
  dry_run: boolean (default: false)
  assets: list of objects (required, min 1 item)
    Each object must contain at minimum:
      external_id: string
      properties: object (deal-specific)

```

Portfolio Declaration Response

PortfolioDeclarationApiResponse:

```
id: UUID
deal_id: UUID
name: string | null
asset_count: integer | null
created_at: datetime
updated_at: datetime
```

Declaration Run Response

DeclarationRunDetailApiResponse:

```
id: UUID
portfolio_declaration_id: UUID
dry_run: boolean
status: "QUEUED" | "PROCESSING" | "COMPLETED" | "COMPLETED_
WITH_ERRORS" | "FAILED"
summary: object | null
error_details: object | null
started_at: datetime | null
completed_at: datetime | null
created_at: datetime | null
updated_at: datetime | null
```

Per-Asset Result Response

DeclarationRunAssetResultApiResponse:

```
asset_id: UUID
external_asset_id: string
contract_external_id: string | null
properties: object | null
is_eligible: boolean | null
is_declarable: boolean | null
results: list of AssetResultApiResponse
    id: UUID
```

```
step_name: string
status: "SUCCESSFUL" | "UNSUCCESSFUL" | "FAILED"
details: object | null
```

Supported Currencies

ALL, ARS, AUD, BGN, BOB, BRL, BZD, CAD, CHF, CLP, CNY, COP, CRC, CZK, DKK, EGP, EUR, GBP, GEL, GTQ, HKD, HUF, IDR, ILS, INR, ISK, JPY, KHR, KRW, LAK, LKR, MAD, MKD, MVR, MXN, MYR, NOK, NZD, PAB, PEN, PHP, PLN, PYG, RON, RSD, SEK, SGD, THB, TRY, TZS, USD, UYU, VND, ZAR

7. Error Handling

HTTP Status Codes

Code	Meaning
200	Success (synchronous operations).
201	Resource created successfully.
202	Accepted — async processing started.
400	Bad request — validation error in request body.
401	Unauthorized — invalid or expired token.
403	Forbidden — you don't have access to this deal/resource.
404	Resource not found.
422	Unprocessable Entity — validation error (e.g., duplicate <code>document_external_id</code> , missing file extension).
500	Internal server error.

Pagination

All list endpoints support pagination:

Parameter	Type	Default	Description
page	int	1	Page number (1-indexed).
size	int	20	Items per page.

Response includes: `total` , `page` , `size` , `pages` .

Idempotency

- **Assets:** `external_id` is the idempotency key. Re-declaring an asset with the same `external_id` performs an **upsert** (updates mutable fields, preserves immutable ones).
- **Payment Transactions:** `external_id` should be unique per transaction.
- **Documents:** `document_external_id` is unique per deal. Re-registering returns `422` ; use Section 4.2 Upload Status to retrieve a fresh presigned URL for any pending document.

Recommended Polling Strategy

For **asset declarations and bulk payment batches**:

1. Poll `GET /v2/declarations/{run_id}` every **10-30 seconds**.
2. Check the `status` field.
3. Once `COMPLETED` or `FAILED` , inspect per-asset results if needed.
4. Typical processing time depends on portfolio size (seconds for small batches, minutes for large portfolios).

For **document uploads**, poll `GET /v2/{deal_id}/documents` (see Section 4.2) every **5-10 seconds** until `uploaded_at` is populated. Each poll on a pending document returns a fresh presigned URL, so if a PUT fails you can retry immediately with the URL from the next poll.